

1 Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat.

2 The Swift Language

Swift is a high-level language originally developed by Apple and then open-sourced in 2015¹. Swift advertises itself as a general purpose language, although one of its most notable uses is in application development for Apple's primary platforms – iOS, macOS, watchOS, and tvOS. In contrast to its predecessor Objective-C, one of Swift's stated goals is safety, and one way that this goal is implemented practically is that Swift has automatic memory management in the form of a reference-counting garbage collector.

2.1 Compiling Swift

To understand how swift's memory management works, we first need to understand the nature of Swift's compilation flow and runtime. The first step of this workflow is parsing and semantic analysis². Swift is a strongly typed language with modern type inference, so these two steps parse the high-level swift code and propagate type information through the AST. Also in this step is the Clang importer, which semantically links clang modules to the code so that they can be part of the type inference and verification.

The next step of compilation is SIL generation. Swift Intermediate Language (SIL), as the name suggests, acts as an intermediary step between high-level swift code and the lower compilation steps. Much like Java bytecode, SIL looks much more like assembly code, with each line (of code) containing an opcode with arguments³. For control flow within functions, SIL uses basic blocks, each of which have their own local variables, arguments and forwarded values for the next basic block. Despite the apparent simplicity of the instructions, it is just as strongly typed than swift, and automatically generates additional verification information such as object lifetimes and ownership information. SIL is also a Static Single Assignment (SSA) language, meaning that each variable is assigned exactly once. At this step of the process, a number of transformations and optimizations are also applied. Relevant to this project, there are actually a number of optimizations focused on reference counting. For the sake of simplicity, I will not be considering these, but they may be worth investigating in the future.

Finally, the SIL is lowered to LLVM IR, yet another intermediate representation, which can then be compiled to a native executable by LLVM.

2.2 The Swift Runtime

Astute readers may notice that at the end of the compilation process, we end up with an executable binary. So why do we need a runtime? And how does the runtime [[attach to]] the program during execution?

What we refer to as the Swift runtime is a section of the Swift standard library which is dynamically

¹<https://www.swift.org/about/>

²<https://www.swift.org/documentation/swift-compiler/>

³<https://github.com/swiftlang/swift/blob/main/docs/SIL/SIL.md>

linked to the executable at runtime⁴. For example, let's see what creating an object looks like.

Source Code	<pre>class A { } let obj = A()</pre>
SIL	<pre>[...] alloc_global @\$s3sil3objAA1ACvp %3 = global_addr @\$s3sil3objAA1ACvp : \$*A %4 = alloc_ref \$A debug_value %4, let, name "self", argno 1 %6 = end_init_let_ref %4 store %6 to %3 [...]</pre>
LLVM IR	<pre>[...] %4 = call noalias ptr @swift_allocObject(ptr %3, i64 16, i64 7) #2 store ptr %4, ptr @"\$s18llvmir_unoptimized3objAA1ACvp", align 8 [...]</pre>

Listing 1: Compilation steps of allocation

We start with some of the simplest swift code that will allocate an object on the heap. In Listing 1 we create a class `A` and instantiate an object `obj`, which will be heap-allocated. `[[WE DO SIL GENERATION]]` and end up with nearly one hundred lines of SIL. A bunch of these lines are autogenerated code for our class `A`. We find two autogenerated `init` functions and two autogenerated `deinit` functions which are `[[implied]]` by our empty class definition. The function we're interested in is the `main()` function, which contains the expanded version of the line `let obj = A()`. Due to SIL being SSA, all of our variables have been renamed, but we can see where they went. For example, note that `@$s3sil3objAA1ACvp` corresponds to our global variable `obj`. The two most important lines in this code are `alloc_ref` and `end_init_let_ref`, which is where we are allocating space for the object on the heap and declaring the object initialized respectively. Since memory is managed by the runtime, we know that this `alloc_ref` instruction must eventually talk to the runtime.

Luckily in the SIL to LLVM IR translation, we get to keep most of the variable names, so finding the equivalent to `alloc_ref` is comparatively simple. I've cut off the allocation of the global variable since it's not what we're interested in, but even still, the latter four lines of SIL are lowered to only two lines of LLVM IR. The first line is what `alloc_ref` is translated into, and it's the runtime hook we're looking for. Specifically, `alloc_ref` is translated to a function call to `@swift_allocObject`, a function defined in the runtime portion of the standard library.

The rest of the Swift runtime is implemented similarly. Memory operations in swift are translated to memory management instructions in SIL which are then lowered to into runtime function calls.

3 Solving Reference Cycles

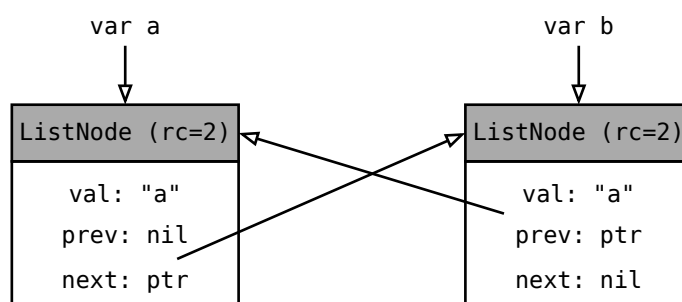
As mentioned earlier, the Swift runtime manages memory via a reference counting garbage collector. `[[upside]]`, reference counting has the pretty serious tradeoff that it cannot detect reference cycles. Consider a doubly linked list defined in Listing 2 using the default "strong" references provided in swift. At the end of this code block, neither `Listnode` object is reachable from our roots (`a` and `b`), but both have a nonzero reference count because they point to each other.

⁴<https://www.swift.org/documentation/standard-library/>

```

class ListNode{
    var val: String
    init(val: String) { self.val = val }
    var next: ListNode?
    var prev: ListNode?
}
var a: ListNode? = ListNode("a")
var b: ListNode? = ListNode("b")
a!.next = b
b!.prev = a
a = nil
b = nil

```

Listing 2: A doubly linked list*Figure 1: A strong reference cycle*

The way that swift solves this is by introducing a number of “weak” pointers—as opposed to the normal “strong” pointers⁵. The three kinds of weak pointers in swift are weak pointers, unowned pointers, and unsafe unowned pointers, which I will refer to as simply “unowned pointers”. All three of these can point to an object without increasing its reference count. What this also means is that all of these weak pointer types may end up pointing to an object whose reference count has reached 0 and therefore has been deallocated. How the three weak pointer types handle that case is their main distinguishing feature:

- Weak pointers must be optional types and, when the referenced object is deallocated, are automatically set to `nil`.
- Unowned pointers panic when a user tries to reference them after deallocation
- Unsafe pointers don’t have any checks, and may allow the user to make use-after-free errors

Unsafe pointers, like the name suggests, violate the memory safety of swift and are therefore highly discouraged. When choosing between the other two options, swift suggests using the concept of ownership and lifetimes. If there are two (or more) objects which may cause a reference cycle, one of them should be designated as the “owner” of the other object and is typically the object with the longer lifetime – the one which is created first and used last. If the owned object has a strictly shorter lifetime than the owner, then it should have a unowned reference, since there’s no risk of panicking. On the other hand, if the owned object might live longer than the owner, weak references should be used.

⁵<https://docs.swift.org/swift-book/documentation/the-swift-programming-language/automaticreferencecounting/>

For example, in our linked list we typically keep a pointer to the head, so we should make the next reference be strong. However, we may want to pop the head of the list and then keep around everything else, so the prev pointer should be weak. In Listing 3 we have the updated class definition, which leads to the references in Figure 2.

```
class ListNode{
  var val: String
  init(val: String) { self.val = val }
  var next: ListNode?
  weak var prev: ListNode?
}
```

Listing 3: An updated doubly linked list

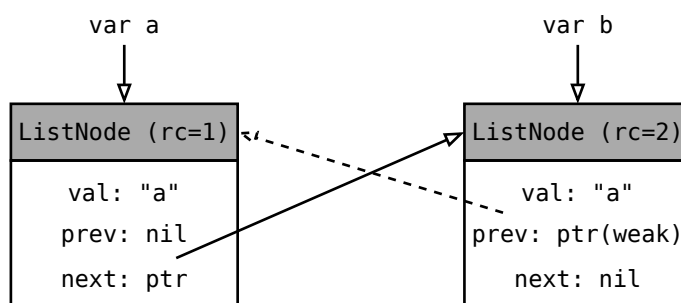


Figure 2: Solving the reference cycle with weak pointers

Once *a* and *b* are unassigned, object B will drop down to 1 reference (from object A next), but object A now has a reference count of 0 since the weak reference from object B doesn't contribute to the count. This leads to object A being deinitialized, removing the reference to object B. This in turn brings B's reference count down to 0, so both objects are correctly deinitialized and deallocated.

3.1 Implementation

Now that we understand how weak pointers work, we can start to understand how they're implemented. Reference counting with only strong references creates a straightforward and elegant system. Each object has one reference count stored as metadata and is deinitialized exactly when its reference count drops to zero. A reference counted runtime system with weak pointers cannot be as simple. For instance, when an unowned pointer tries to access an object with no more strong references, it needs to panic. How does it "know" that the object has been deinitialized? Presumably there's a flag somewhere, but now that's extra state that we didn't need to store previously. And now *that* state needs to be deallocated somehow when there are no more unowned pointers.

Since unowned pointers are the simpler of the two, let's start with looking at their implementation. Rather than containing a basic reference count in the object's header, each heap-allocated `HeapObject` in Swift contains a `InlineRefCounts` struct, containing a strong reference count, an unowned reference count, and the object's "state", which follows the state machine in Figure 3. We abbreviate strong reference count as SRC, unowned reference count as URC, and (when we add them in) weak reference count as WRC.

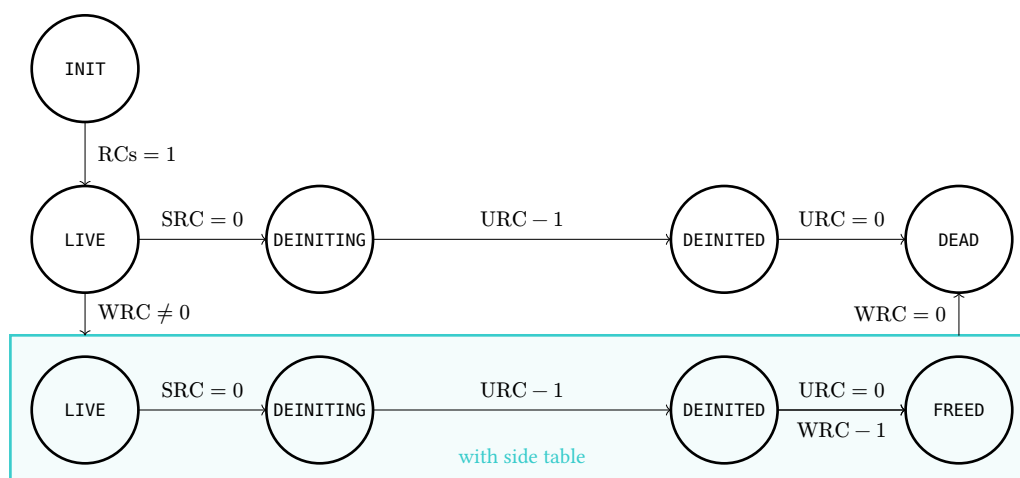


Figure 3: The object lifecycle state machine

An object's normal state is **LIVE**, where all operations are valid. Once the strong reference count reaches zero, instead immediately deallocating the object, we move to the **DEINITING** and call the user-specified deinit function. During this time, attempting to dereference an unowned reference will result in an error. The auto-generated deinit function also removes all the references inside the object during this stage. Once the deinit function finishes, the object moves to the **DEINITED** state and the unowned reference count is decremented. Note that the **URC** starts at 1, so the unowned reference count now accurately counts the number of unowned references. Unowned references to the object may still exist at this time, but any attempt to dereference them will result in a panic. Once the unowned reference count reaches 0 (which may be immediately if there are no unowned references to the object), the object is actually freed.

Based on this implementation, we can see that unowned references have relatively similar performance overhead to strong references. Each assignment and deassignment requires updating a reference count, and then we have some extra logic when either reference count hits zero. However, unowned reference have a major issue in terms of memory. If we create some large object **A** and then remove all of the strong references to it but keep an unowned reference around, all the space we allocated for the object must still exist.

Weak references solve this issue through a structure called the side table. An object's side table entry is allocated separately from the main object, and contains all three reference counts (strong, weak, and unowned). The side table and main object also contain mutual pointers to each other. Unlike strong and unowned references which point directly to the object, weak references are internally pointers to the side table.

Initially, objects start with no side table, and only gain one when a weak reference is created. This corresponds to the bottom path of the object life cycle shown in Figure 3. While very similar to the top path, there are a few key differences. First, all reference counts are moved into the side table. Next, the object gains a weak reference count. Similar to the unowned reference count, this starts out as 1 + the real number of weak references.

On the “with side table” path, when a **DEINITED** object's unowned reference count drops to 0, it moves to a new state: **FREED**. In this transition, the original object's memory is freed—leaving only the side

table entry—and the weak reference count is decremented to come in line with the real number of weak references. In the FREED state, only weak references to the object should remain, meaning all pointers to the original object's memory location should be dropped. When a weak reference is checked, swift only need check the side table's state before returning either a nil value or a strong pointer.

Finally, when the weak reference count drops to 0, the side table entry is deallocated, and the object can move to DEAD.

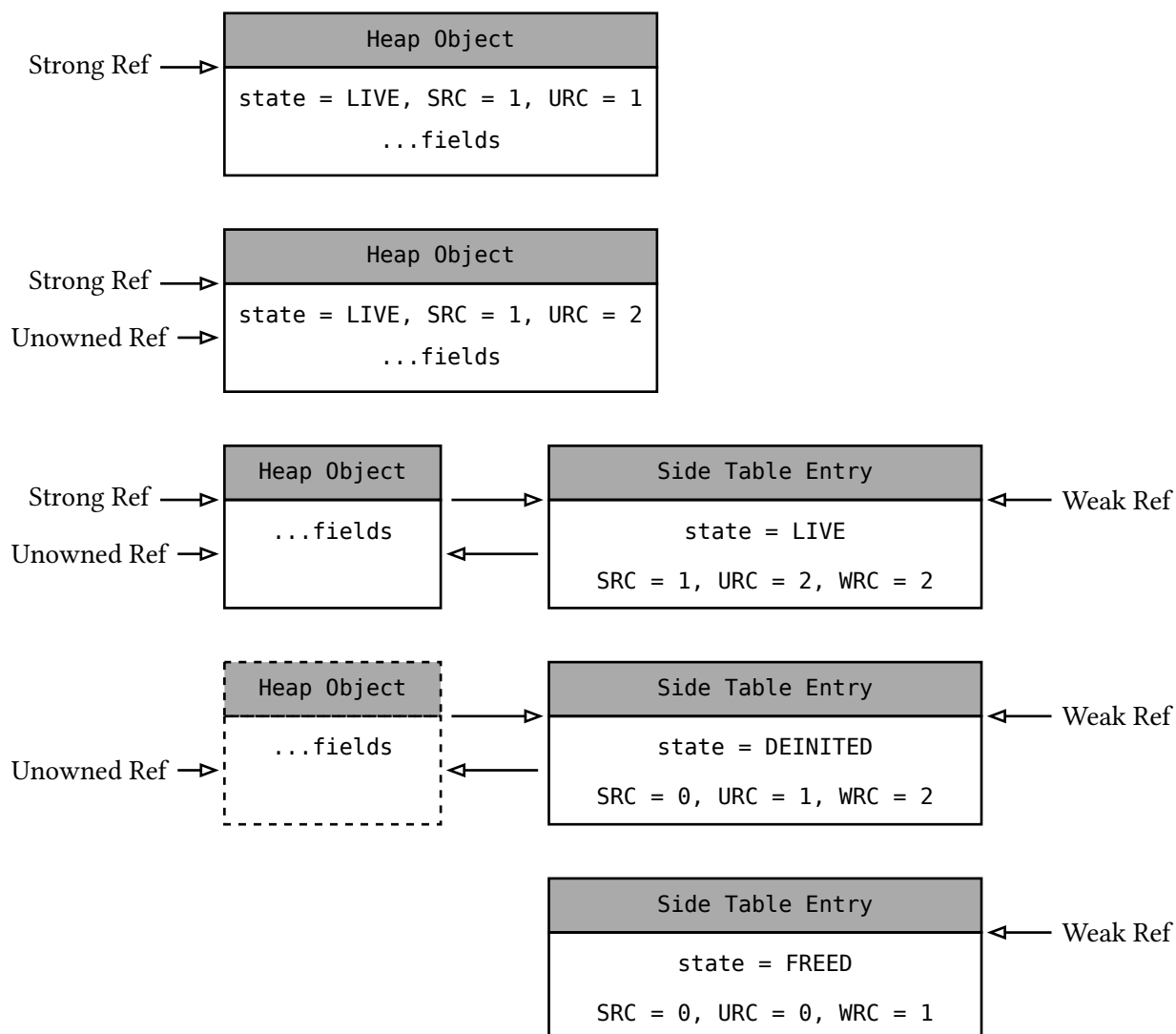


Figure 4: An object following the side table lifecycle

Above, Figure 4 shows the progression of an object gaining a strong, unowned, then weak reference and then losing them again in that order.

4 Performance

Based on the implementation described above, we can see that there are some performance tradeoffs for using strong pointers. For example, the first weak pointer to an must allocate a second segment of heap

memory, which takes significant time. To test this I wrote three benchmarks to test reference creation, deinitialization and dereference, as described in Listing 4.

Classes (strong ref) <pre> class A{} class B{ var ref: A init(a: A){ self.ref = a } } </pre>	Create <pre> let a = A() for _ in 0..<count <="" _="B(a:" a)="" let="" pre="" {="" }=""> </count></pre>
Deref <pre> let a = A() let b = B(a: a) for _ in 0..<count <="" _="b.ref" let="" pre="" {="" }=""> </count></pre>	Destroy <pre> var b: B for _ in 0..<count <="" a="A()" a)="" b="B(a:" let="" pre="" {="" }=""> </count></pre>

Listing 4: Benchmarking code

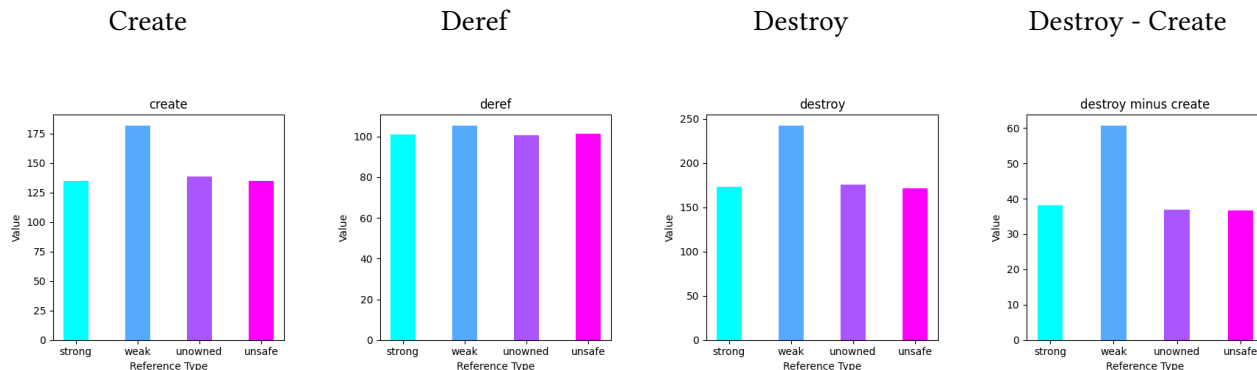


Figure 5: Benchmarking performance results

Each benchmark was run with `B.ref` set to all four reference types, run simultaneously to control for external factors affecting performance. The results for $N = 1,000,000,000$ are described in Figure 5. For the “Destroy” test, I specifically wanted to test the performance of moving from the LIVE state through the state machine to DEAD. Since testing this necessarily also involved creating a reference, I also plotted the difference of the destroy and create tests to see if the performance difference could be accounted for by the creation time involved.

These times reflect what we expect from the implementation. Creating a strong, unowned, or unsafe reference take the same amount of time since they behave very similarly. Creating a weak

pointer allocates the side table entry, so it takes significantly longer. Similar for Destroy, the strong, unowned, and unsafe cases all are moving through the DEINITING, DEINITED and DEAD states, albeit at different points in execution, so we expect them to all take a similar amount of time. Weak pointers must move through the FREED state and also deallocate the side table, so they take more time, even when the extra creation time is factored out.

The most surprising result to me personally is that the dereference time is similar across all reference types. I would expect that weak pointers would be slightly slower since they need to load the side table and then load the actual object based on the side table entry's pointer. However, what the benchmarking shows is that they are the slowest, but by a nearly undetectable amount.

5 Process

At the start of this project, I had some prior experience with Swift, but hadn't fully explored the language. I chose Swift because it was a language that was created with mobile development in mind and I wanted to see how that affected the design of the runtime.

In my first week of research (week 5 of the quarter) I had a few goals: re-learn swift as a language, and find out where I could find information on the Swift runtime. I followed A Swift Tour⁶ and started learning about the compilation pipeline. During this time I also found a very informative blog post by Jacob's Tech Tavern⁷ about how memory operations turned into runtime calls. Both the compilation pipeline and the blog post referenced SIL, so for the tail end of this week I started reading the SIL documentation.

I underestimated how dense the SIL documentation would be, so for the entirety of week 6, I was just learning SIL. At the start of week 7, I just the last part of the documentation I hadn't finished, and returned to the blog post. The post itself is quite short, but it references lots of different parts of the swift repository, so I spent rest of week 7 following those links and getting a feel for how the repo is structured. Also during this time I started looking into how swift compiled for different platforms to try and find information on mobile-specific considerations or optimizations. What I found was that platform specifications were handled via a lot of compile-time macros and therefore were quite hard to understand. In addition, the constants used to activate these macros didn't have great documentation on when they were activated, so finding exactly which code ran on IOS vs OSX was even more opaque.

Week 8 was a very productive week and also the week where my focus changed to weak references. I first read the Swift's Automatic Reference Counting documentation⁸ at the start of this week, and seriously misunderstood how the different reference types worked. After doing some of my own testing, I corrected most of my initial misunderstandings, but also started wondering about the performance implications of these references. Using what I had learned from my SIL research and the process outlined in the Jacob's Tech Tavern article, I began to trace a number of unowned reference operations. Again I ran into issues with C++ macros, specifically that certain function names were defined via macro. For example, when I looked for where `visitLoadUnownedInst` was defined, I couldn't find anything because it was defined as `visitLoad##Name##Inst` in the `NEVER_LOADABLE_CHECKED_REF_STORAGE` macro.

⁶<https://docs.swift.org/swift-book/documentation/the-swift-programming-language/guidedtour/>

⁷<https://blog.jacobstechtavern.com/p/the-swift-runtime-your-silent-partner>

⁸<https://docs.swift.org/swift-book/documentation/the-swift-programming-language/automaticreferencecounting/>

Week 9 was a continuation of the previous weeks work, finally getting all the way down into the Swift runtime after chasing function calls for a while. All of my calls seemed to be ending in `HeapObject` calls, but my lack of experience with C++ headers got me a bit turned around as to where exactly the `HeapObject` calls were implemented.

At the beginning of week 10 I wanted to have some solid results for the in-class presentation, so I switched my focus to creating the benchmarks. They were relatively straightforward, but took some time to tune and mitigate interference from other processes on my computer. After creating and giving the presentation, I returned to the source code investigation with fresh eyes and almost immediately found the relevant portion of `HeapObject.h` and `RefCount.h`. Between these two files I was able to fully construct the reference counting model presented in this report.

6 Future Work

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat.